

COMPUTATIONAL METHODS LAB MANUAL

SEMESTER V

**DIVISION OF
MECHANICAL ENGINEERING**

**SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND
TECHNOLOGY
CUSAT**

Syllabus

Review of fundamentals of C programming, Pointers-pointer declaration-pointers and one dimensional arrays-pointers and functions, Data files- opening and closing a data file-creating a data file- processing a data file. C-graphics-drawing lines, rectangles, circles and ellipse

Numerical Techniques: Preparation of computer programs for solution of polynomial and transcendental equations: bisection method, regula falsi method, successive iteration- Newton Raphson method. Solution of system linear algebraic equations : Gauss elimination- matrix inversion, Gauss Jordan method, Gauss-Seidel method.

Numerical integration : trapezoidal rule- Simpson's $1/3$ rule- Gauss quadrature formulae

Numerical solution of ordinary differential equations : Taylor series method- Runge- kutta method Numerical solution of boundary value problems.

CONTENTS

Sl. No.	TITLE	Page No.
1	BISECTION METHOD	1
2	REGULA FALSI METHOD	9
3	SUCCESSIVE ITERATION METHOD	16
4	NEWTON-RAPHSON METHOD	21
5	GAUSS ELIMINATION METHOD	26
6	GAUSS-JORDAN METHOD	35
7	GAUSS-SEIDEL METHOD	39
8	TRAPEZOIDAL RULE	46
9	SIMPSON'S RULE	54
10	GAUSS QUADRATURE FORMULAE	61
11	TAYLOR'S SERIES METHOD	67
12	RUNGE-KUTTA METHOD	71

BISECTION METHOD

One of the first numerical methods developed to find the root of a nonlinear equation $f(x) = 0$ was the bisection method (also called *binary-search* method). The method is based on the following theorem.

Theorem

An equation $f(x) = 0$, where $f(x)$ is a real continuous function, has at least one root between x_l and x_u if $f(x_l)f(x_u) < 0$ (See Figure 1).

Note that if $f(x_l)f(x_u) > 0$, there may or may not be any root between x_l and x_u (Figures 2 and 3). If $f(x_l)f(x_u) < 0$, then there may be more than one root between x_l and x_u (Figure 4). So the theorem only guarantees one root between x_l and x_u .

Bisection method

Since the method is based on finding the root between two points, the method falls under the category of bracketing methods.

Since the root is bracketed between two points, x_l and x_u , one can find the mid-point, x_m between x_l and x_u . This gives us two new intervals

1. x_l and x_m , and
2. x_m and x_u .

Is the root now between x_l and x_m or between x_m and x_u ? Well, one can find the sign of $f(x_l)f(x_u)$, and if $f(x_l)f(x_u) < 0$ then the new bracket is between x_l and x_m , otherwise, it is between x_m and x_u . So, you can see that you are literally halving the interval. As one repeats this process, the width of the interval $[x_l, x_u]$ becomes smaller and smaller, and you can zero in to the root of the equation $f(x) = 0$.

Advantages of bisection method

- a) The bisection method is always convergent. Since the method brackets the root, the method is guaranteed to converge.
- b) As iterations are conducted, the interval gets halved. So one can guarantee the error in the solution of the equation.

Drawbacks of bisection method

- a) The convergence of the bisection method is slow as it is simply based on halving the interval.
- b) If one of the initial guesses is closer to the root, it will take larger number of iterations to reach the root.
- c) If a function $f(x)$ is such that it just touches the x-axis (Figure 6) such as $f(x) = x^2 = 0$ will be unable to find the lower guess, x_l , and upper guess, x_u , such that $f(x_l)f(x_u) < 0$
- d) For functions $f(x)$ where there is a singularity and it reverses sign at the singularity, the bisection method may converge on the singularity (Figure 7).

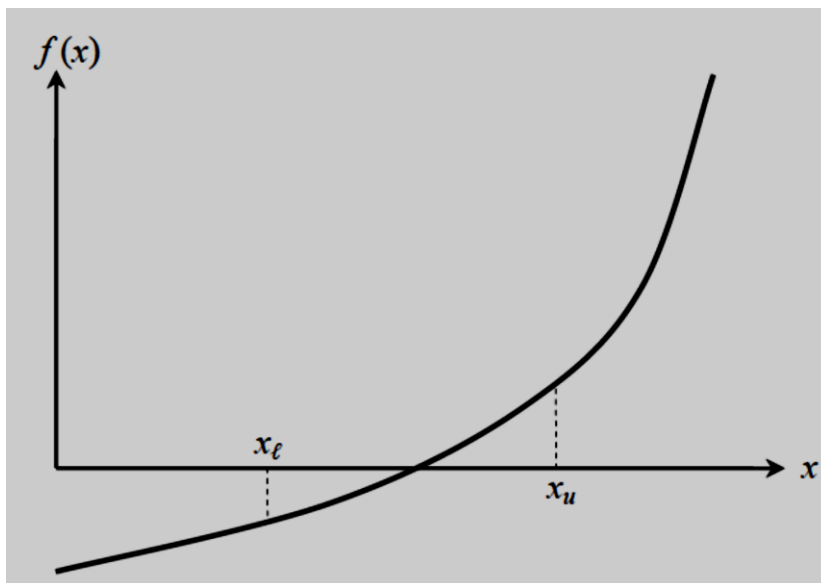


Figure 1 At least one root exists between the two points if the function is real, continuous, and changes sign.

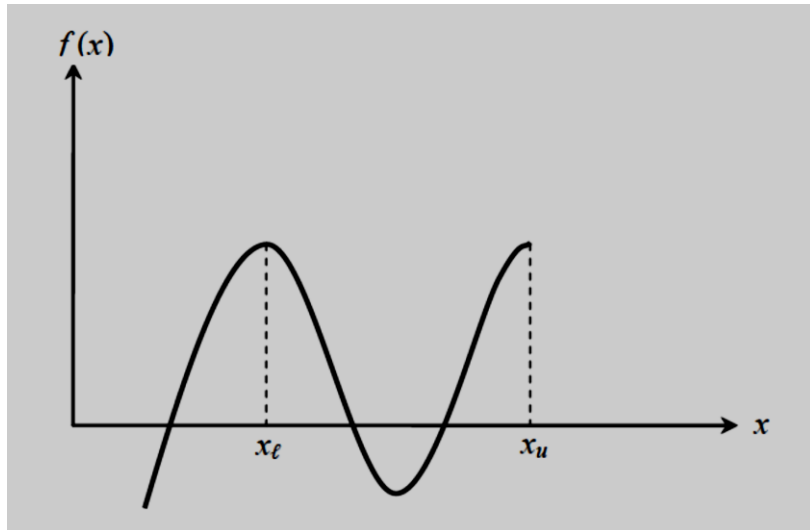


Figure 2 If the function $f(x)$ does not change sign between the two points, roots of the equation $f(x) = 0$ may still exist between the two points.

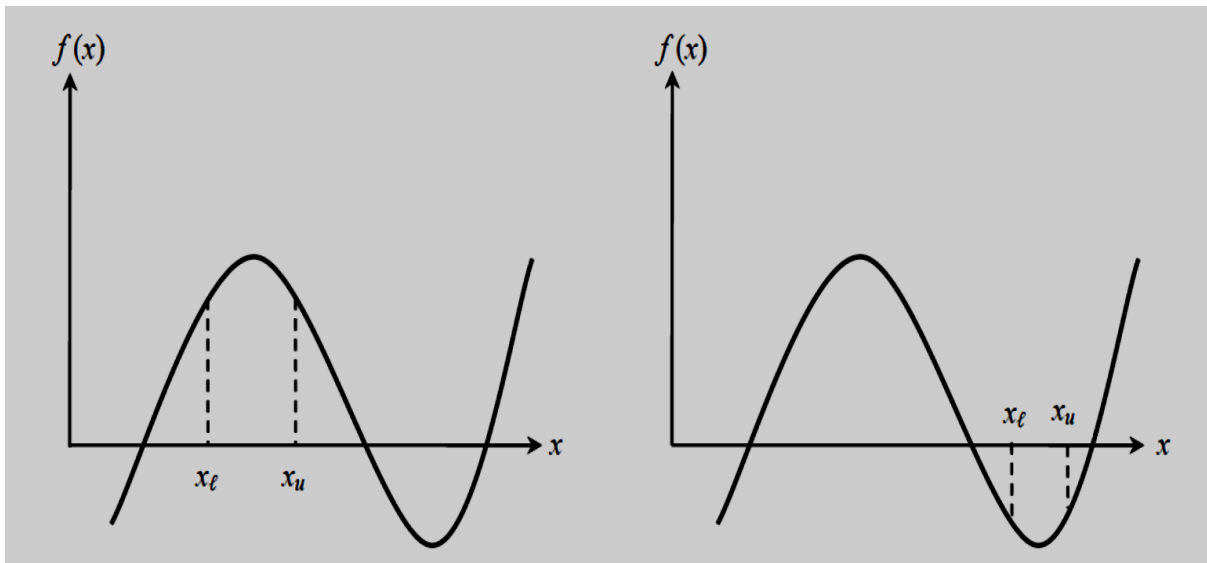


Figure 3 If the function $f(x)$ does not change sign between two points, there may not be any roots for the equation $f(x) = 0$ between the two points.

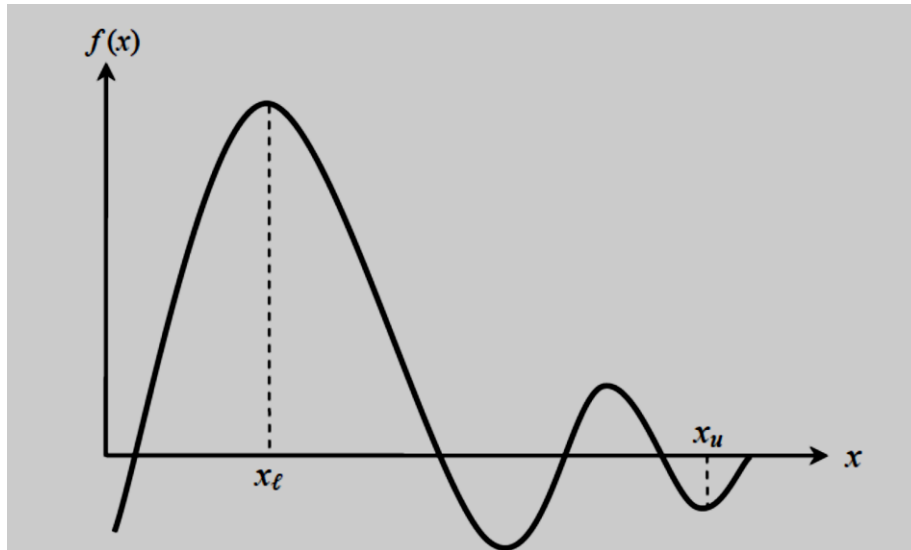


Figure 4 If the function $f(x)$ changes sign between the two points, more than one root for the equation $f(x) = 0$ may exist between the two points.

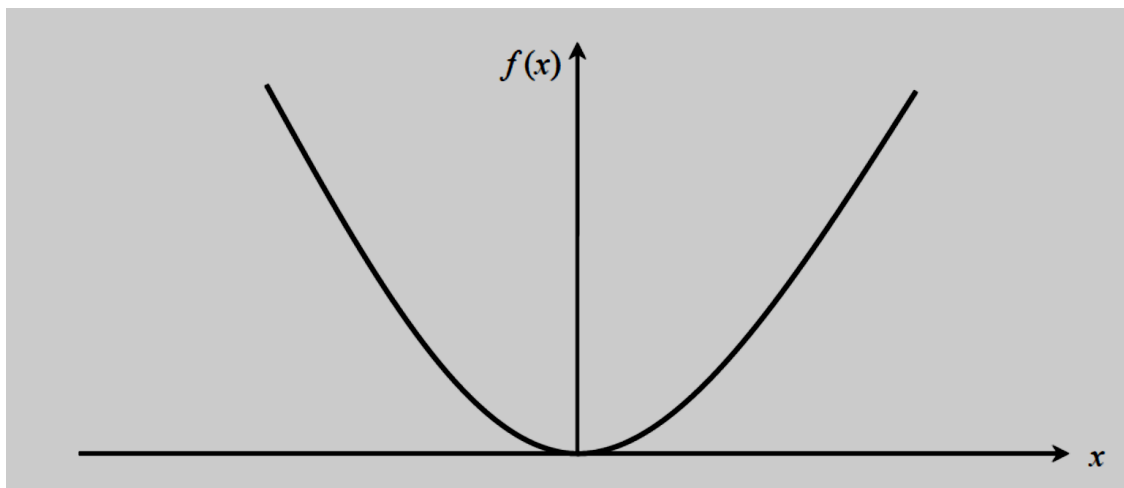
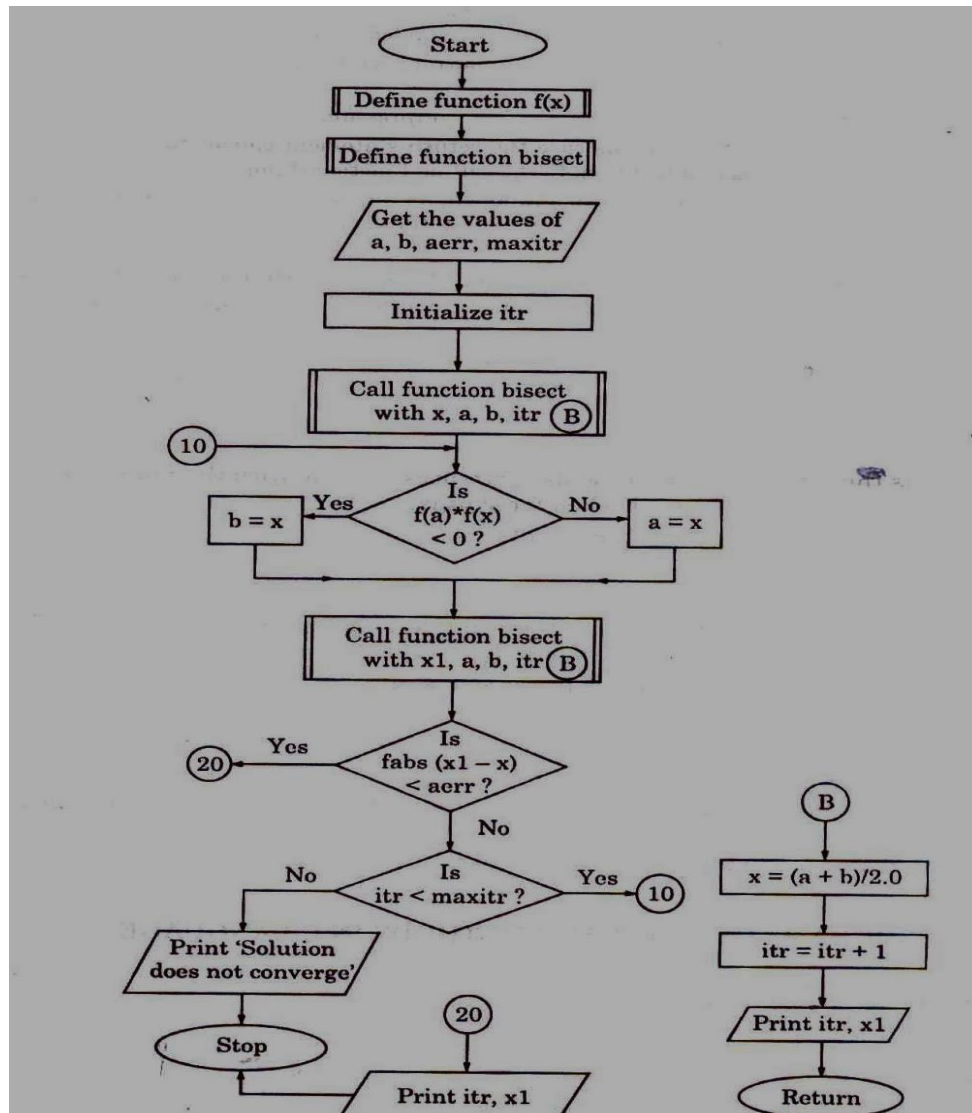


Figure 6 The equation $f(x) = x^2 = 0$ has a single root at $x = 0$ that cannot be bracketed.

ALGORITHM



SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<math.h>

void bisect(float a,float b, int noit,float acc)

{

float F(float);

float x1,x2,x0,f1,f2,f0,root;

x1=a;

x2=b;

f1=F(x1);          //Compute f1=f(x1) & f2=f(x2)

f2=F(x2);

//If f1*f2 > 0.0 print message and exit

//otherwise do the iterations

if((f1*f2)>0.0){printf("\nRoots are not bracketed");return;}

{

int i=1;

while(i<noit)

    {

        x0=(x1+x2)*0.5;    //Root is initialized as (x1+x2)/2.0

        f0=F(x0);

        if(f0==0){root=x0; printf("\n Root= %f",root);return;}

        if((f1*f0)<0.0)    //f1 is compared with fo

```

```

        {           //If f1*f0 is less than zero root lies
x2=x0;           // between x0 and x1 and x2 is replaced 0
        }           // with x
else
        {           //If f1*f0 is greater than zero root lies
x1=x0;           //between x0 and x2 and x1 is replaced
f1=f0;           //with x0
        }

//Exit if abs(x2-x1) is less than the specified limit for accuracy
if(fabs(x2-x1)<acc){printf("\nRoot = %f",(x1+x2)*0.5);return;}

i=i+1;
    }
}

//Function definition
float F(float x)
{
float y;
y=x*x-4.0*x-10.0;
return(y);
}

main()
{
clrscr();           //Clear the screen

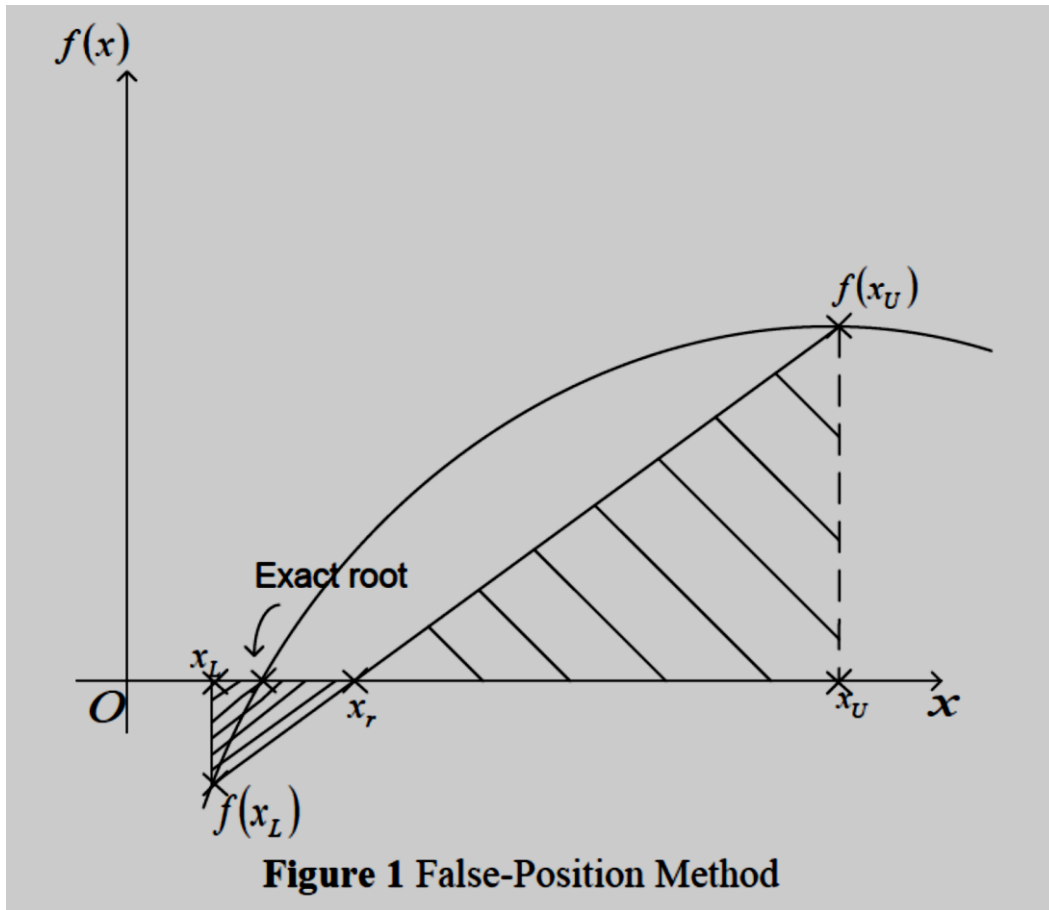
```

```
float a,b,acc;int noit;  
a=-2.0;b=-1.0;acc=0.0001;noit=30;//Decide initial values for x1&x1  
                                //and stopping criterion.  
bisect(a,b,noit,acc);          //Invoke function bisect.  
while(!kbhit());  
}
```

REGULA FALSI METHOD

The bisection method was described as one of the simple bracketing methods of solving a nonlinear equation of the general form

$$f(x) = 0 \quad (1)$$



The above nonlinear equation can be stated as finding the value of x such that Equation (1) is satisfied.

In the bisection method, we identify proper values of x_L (lower bound value) and x_U (upper bound value) for the current bracket, such that

$$f(x_L) f(x_U) < 0. \quad (2)$$

The next predicted/improved root x_r can be computed as the midpoint between x_L and x_U as

$$x_r = \frac{x_L + x_U}{2} \quad (3)$$

The new upper and lower bounds are then established, and the procedure is repeated until the convergence is achieved (such that the new lower and upper bounds are sufficiently close to each other).

However, in the example shown in Figure 1, the bisection method may not be efficient because it does not take into consideration that $f(x_L)$ is much closer to the zero of the function $f(x)$ as compared to $f(x_U)$. In other words, the next predicted root x_r would be closer to x_L (in the example as shown in Figure 1), than the mid-point between x_L and x_U . The false-position method takes advantage of this observation mathematically by drawing a secant from the function value at x_L to the function value at x_U , and estimates the root as where it crosses the x-axis.

False-Position Method

Based on two similar triangles, shown in Figure 1, one gets

$$\frac{0 - f(x_L)}{x_r - x_L} = \frac{0 - f(x_U)}{x_r - x_U} \quad (4)$$

From Equation (4), one obtains

$$\begin{aligned} (x_r - x_L)f(x_U) &= (x_r - x_U)f(x_L) \\ x_U f(x_L) - x_L f(x_U) &= x_r \{f(x_L) - f(x_U)\} \end{aligned}$$

The above equation can be solved to obtain the next predicted root x_m as

$$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)} \quad (5)$$

The above equation, through simple algebraic manipulations, can also be expressed as

$$x_r = x_U - \frac{f(x_U)}{\left\{ \frac{f(x_L) - f(x_U)}{x_L - x_U} \right\}} \quad (6)$$

or

$$x_r = x_L - \frac{f(x_L)}{\left\{ \frac{f(x_U) - f(x_L)}{x_U - x_L} \right\}} \quad (7)$$

False-Position Algorithm

The steps to apply the false-position method to find the root of the equation $f(x)=0$ are as follows.

1. Choose x_L and x_U as two guesses for the root such that $F(x_L)f(x_U) < 0$, or in other words, $f(x)$ changes sign between x_L and x_U .
2. Estimate the root, of the equation $f(x)=0$ as

$$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)}$$

3. Now check the following

If $f(x_L)f(x_r) < 0$, then the root lies between x_L and x_r ; then $x_L = x_L$ and $x_U = x_r$.
 If $f(x_L)f(x_r) > 0$, then the root lies between x_r and x_U ; then $x_L = x_r$ and $x_U = x_U$.
 If $f(x_L)f(x_r) = 0$, then the root is x_r . Stop the algorithm.

4. Find the new estimate of the root

$$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)}$$

Find the absolute relative approximate error as

$$|\epsilon_a| = \left| \frac{x_r^{new} - x_r^{old}}{x_r^{new}} \right| \times 100$$

where

x_r^{new} = estimated root from present iteration

x_r^{old} = estimated root from previous iteration

5. Compare the absolute relative approximate error $|\epsilon_a|$ with the pre-specified relative error tolerance ϵ_s . If $|\epsilon_a| < \epsilon_s$, then go to step 3, else stop the algorithm. Note one should also check whether the number of iterations is more than the maximum number of iterations allowed. If so, one needs to terminate the algorithm and notify the user about it.

Note that the false-position and bisection algorithms are quite similar. The only difference is the formula used to calculate the new estimate of the root x_r as shown in steps #2 and #4!

SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<math.h>

#include<stdlib.h>

void regufal(float (*fun)(float),float a,float b,float acc,int itno)
{
float fl,fh,xl,xh,temp,x0,f0,del; int i;

fl=(*fun)(a);
fh=(*fun)(b);
if(fl*fh>0.0){printf("\nRoot is not bracketed");}

if(fl<0.0) {
                xl=a;
                xh=b;
            }
else
            {
                xl=b;
                xh=a;
                temp=fl;
                fl=fh;
                fh=temp;
            }

```



```

i=1;
while(i<itno)
{
x0=xl-(xh-xl)*fl/(fh-fl);
f0=(*fun)(x0);
if(f0<0.0){
                del=xl-x0;
                xl=x0;
                fl=f0;
            }
else
            {
                del=xh-x0;
                xh=x0;
                fh=f0;
            }

if(fabs(del)<acc){printf("\nRoot= %f\nNo.Iterations=%d",x0,i);return ;}
else
{i=i+1;}
}
printf("\nNo. of iterations not sufficient/root not converging");
}

float F(float x)
{

```

```
float y;  
y=x*x*x-2.0*x-5.0;  
return y;  
}  
main()  
{  
clrscr();  
float a,b,acc;int n;  
a=-1.0;b=3.0;n=130;acc=0.0001;  
regufal(F,a,b,acc,n);  
}
```

SUCCESSIVE ITERATION METHOD

Consider the system $Ax = b$, with $A = D - L - U$.

where D is a diagonal matrix with elements a_{ii} ,

L is a strictly lower triangular matrix, $L = [l_{ij}]$ such that

$$l_{ij} = \begin{cases} -a_{ij}, & i > j \\ 0, & i \leq j, \end{cases}$$

and U is a strictly upper triangular matrix, $U = [u_{ij}]$ such that

$$u_{ij} = \begin{cases} -a_{ij}, & i < j \\ 0, & i \geq j. \end{cases}$$

When trying to solve $Ax = b$, we obtain an approximate solution $x^{(k)}$ of the true solution x . The quantity $r^{(k)} = b - Ax^{(k)}$ is called a *residual* and it is a measure of the accuracy of $x^{(k)}$. Clearly, we would like to make the residual $r^{(k)}$ to be as small as possible for each approximate solution $x^{(k)}$.

Now remember, when calculating $x_i^{(k)}$, the components $x_1^{(k+1)}, \dots, x_{(i-1)}^{(k+1)}$ are already known. So in the Gauss-Seidel iterative method for the most recent approximation, the residual vector is given by

$$r^{(k)} = b - Dx^{(k)} + Lx^{(k+1)} + Ux^{(k)}.$$

Ultimately, we wish to make $x - x^{(k)}$ as small as possible. However, as we don't know x yet, we instead consider $x^{(k+1)} - x^{(k)}$ as a measure for $x - x^{(k)}$. We now wish to calculate $x^{(k+1)}$ such that

$$D(x^{(k+1)} - x^{(k)}) = \omega(b - Dx^{(k)} + Lx^{(k+1)} + Ux^{(k)}),$$

where ω is called the relaxation parameter. Re-arranging, we get

$$(\mathbf{D} - \omega\mathbf{L})\mathbf{x}^{(k+1)} = ((1 - \omega)\mathbf{D} + \omega\mathbf{U})\mathbf{x}^{(k)} + \omega\mathbf{b},$$

and hence, the recurrence relation is given by

$$\mathbf{x}^{(k+1)} = (\mathbf{D} - \omega\mathbf{L})^{-1}((1 - \omega)\mathbf{D} + \omega\mathbf{U})\mathbf{x}^{(k)} + (\mathbf{D} - \omega\mathbf{L})^{-1}\omega\mathbf{b}.$$

The process of reducing residuals at each stage is called “**Successive Relaxation**”. If $0 < \omega < 1$, the iterative method is known as a “**Successive Under Relaxation**” and they can be used to obtain convergence when the Gauss-Seidel scheme is not convergent. For choices of $\omega > 1$ the scheme is a “**Successive Over-Relaxation**” and is used to accelerate convergent Gauss-Seidel iterations. Note, $\omega = 1$ is simply the Gauss-Seidel Iterative Method.

The iteration matrix for the **S.O.R. method - Successive Over-Relaxation** with $\omega > 1$ is given by

$$\mathbf{B}_{\text{SOR}} = (\mathbf{D} - \omega\mathbf{L})^{-1} [(1 - \omega)\mathbf{D} + \omega\mathbf{U}].$$

The iteration matrix $\mathbf{B}_{\text{SOR}}$ can be derived by splitting $\mathbf{A}$ in the following way:

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U} = \mathbf{D} \left(1 - \frac{1}{\omega}\right) + \frac{1}{\omega}\mathbf{D} - \mathbf{L} - \mathbf{U}, \quad \omega > 0.$$

Thus $\mathbf{Ax} = \mathbf{b}$ can be written as

$$\begin{aligned} \left(\frac{1}{\omega}\mathbf{D} - \mathbf{L}\right)\mathbf{x} &= \left(-\left(1 - \frac{1}{\omega}\right)\mathbf{D} + \mathbf{U}\right)\mathbf{x} + \mathbf{b} \\ (\mathbf{D} - \omega\mathbf{L})\mathbf{x} &= ((1 - \omega)\mathbf{D} + \omega\mathbf{U})\mathbf{x} + \omega\mathbf{b}, \end{aligned}$$

so,

$$\mathbf{B}_{\text{SOR}} = (\mathbf{D} - \omega \mathbf{L})^{-1} [(1 - \omega) \mathbf{D} + \omega \mathbf{U}]$$

The aim is to choose ω such that the rate of convergence is maximised, that is the spectral radius, $\rho(\mathbf{B}_{\text{SOR}}(\omega))$, is minimised. How do we find the value of ω that does this? There is no complete answer for general $N \times N$ systems, but it is known that if, for each, $1 \leq i \leq N$, $a_{ii} \neq 0$ then

$$\rho(\mathbf{B}_{\text{SOR}}) \geq |1 - \omega|.$$

This means that for convergence we must have $0 < \omega < 2$.

SAMPLE PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<math.h>

#include<stdlib.h>

float regufal(float (*fun)(float), float a,float b,float acc,int itno)
{
float fl,fh,xl,xh,temp;

fl=(*fun)(a);
fh=(*fun)(b);

if(fl*fh<0.0){printf("\n Root is not bracketed");return;}

{
if(fl<0.0){
x1=a;
xh=b;
}
else {
```

```

        x1=x2;

        xh=x1;

        temp=fl;

        fl=fh;

        fh=temp;

float x1;

int i=0;

while(i<n)

{

x1=(*fun)(x0);

if(fabs(x1-x0)<acc){printf("\n Root = %f\n No. iterations =%d",x1,i);return;}

x0=x1;

i=i+1;

}

printf("\n No of iteration not sufficient/the root is not converging");

}

float F(float x)

{

float y;

y=0.5*(cos(x)+3.0);

return y;

}

main()

{

```

```
clrscr();  
float x0,acc;int n;  
x0=3.14/2.0,acc=.0001,n=30;  
iter(F,x0,acc,n);  
}
```

NEWTON-RAPHSON METHOD

Methods such as the bisection method and the false position method of finding roots of a nonlinear equation $f(x) = 0$ require bracketing of the root by two guesses. Such methods are called *bracketing methods*. These methods are always convergent since they are based on reducing the interval between the two guesses so as to zero in on the root of the equation. In the Newton-Raphson method, the root is not bracketed. In fact, only one initial guess of the root is needed to get the iterative process started to find the root of an equation. The method hence falls in the category of *open methods*. Convergence in open methods is not guaranteed but if the method does converge, it does so much faster than the bracketing methods.

Derivation

The Newton-Raphson method is based on the principle that if the initial guess of the root of $f(x) = 0$ is at x_i , then if one draws the tangent to the curve at $f(x_i)$, the point x_{i+1} where the tangent crosses the x-axis is an improved estimate of the root (Figure 1).

Using the definition of the slope of a function, at $x = x_i$

$$\begin{aligned} f'(x_i) &= \tan \theta \\ &= \frac{f(x_i) - 0}{x_i - x_{i+1}}, \end{aligned}$$

which gives

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (1)$$

Equation (1) is called the Newton-Raphson formula for solving nonlinear equations of the form $f(x) = 0$. So starting with an initial guess, x_i , one can find the next guess, x_{i+1} , by using Equation (1). One can repeat this process until one finds the root within a desirable tolerance.

Drawbacks of the Newton-Raphson Method

1. Divergence at inflection points

If the selection of the initial guess or an iterated value of the root turns out to be close to the inflection point (see the definition in the appendix of this chapter) of the function $f(x)$ in the equation $f(x) = 0$, Newton-Raphson method may start diverging away from the root. It may then start converging back to the root.

2. Oscillations near local maximum and minimum

Results obtained from the Newton-Raphson method may oscillate about the local maximum or minimum without converging on a root but converging on the local maximum or minimum. Eventually, it may lead to division by a number close to zero and may diverge.

3. Root jumping

In some case where the function $f(x)$ is oscillating and has a number of roots, one may choose an initial guess close to a root. However, the guesses may jump and converge to some other root.

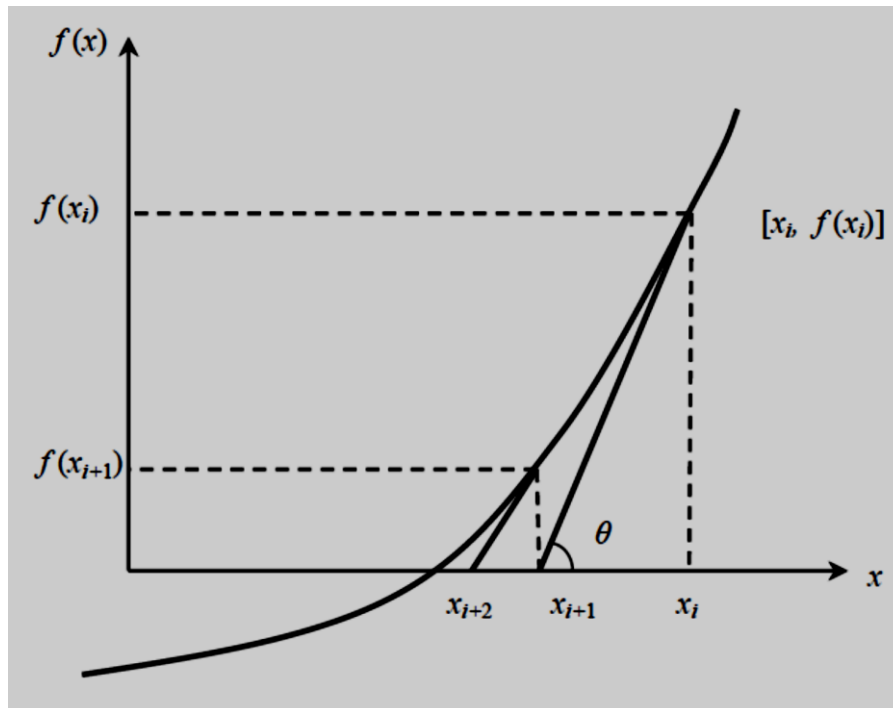
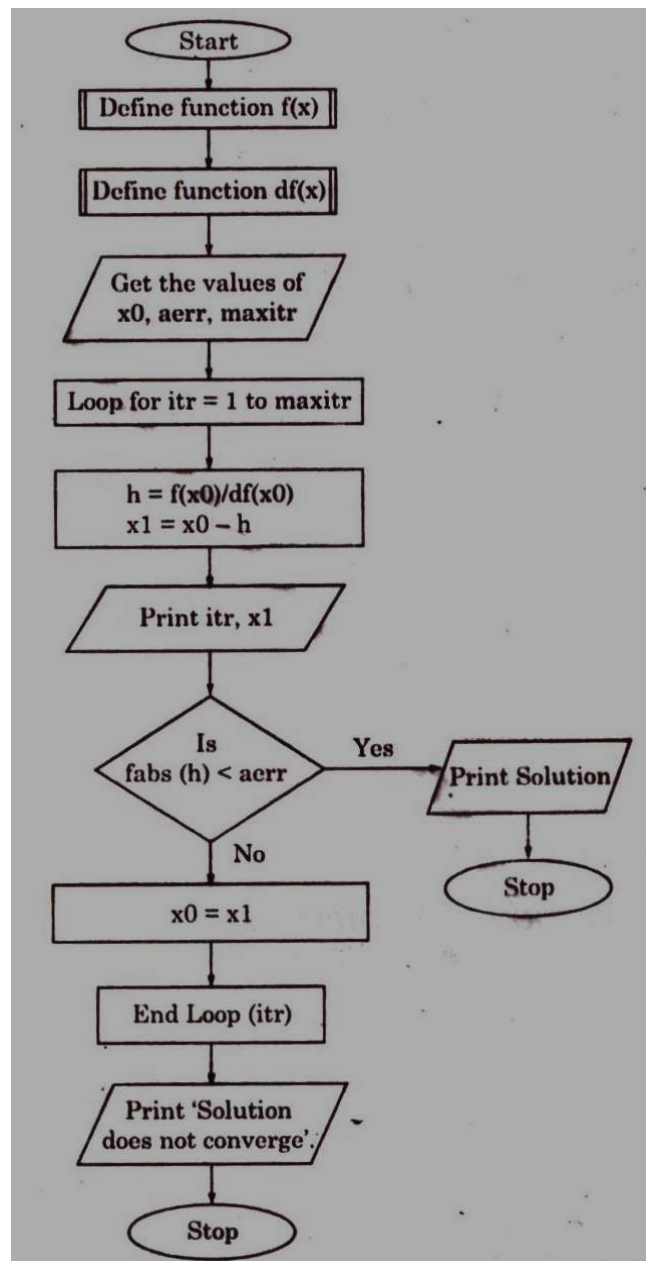


Figure 1 Geometrical illustration of the Newton-Raphson method.

ALGORITHM



SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<math.h>

#include<stdlib.h>

//void (*fun)(float ,float *,float *) returns a pointer to a function which
//return value of the function F and value of its derivative dF

void newton(void (*fun)(float ,float *,float *),float x0,float acc,int itno)
{
float x1,F,dF ;
int i=1;
while(i<itno)
{
(*fun)(x0,&F,&dF); //This function call fetch F&dF
x1=x0-F/dF;      //From x0 refined value x1 is obtained
if(fabs(x1-x0)<acc){printf("\n Root = %f",x1);return;}
x0=x1;
i=i+1;
}
printf("\n No of Iterations are not sufficient/ Root is not converging");
}

void fun(float x,float *F,float *dF)
{
*F =x*sin(x)+cos(x); //Equation of which the root is to be determined

```

```
*dF=x*cos(x);    //Derivative of the equation
}
void main()
{
clrscr();
float x0=3.140,acc=.01;int n=30;
float *F,*dF;
newton(fun,x0,acc,n);//Invoke the function newton.
getch();
}
```

GAUSS ELIMINATION METHOD

One of the most popular techniques for solving simultaneous linear equations is the Gaussian elimination method. The approach is designed to solve a general set of n equations and n unknowns

$$\begin{array}{l} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n = b_2 \\ \cdot \qquad \qquad \qquad \cdot \\ \cdot \qquad \qquad \qquad \cdot \\ \cdot \qquad \qquad \qquad \cdot \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \dots + a_{nn}x_n = b_n \end{array}$$

Gaussian elimination consists of two steps

1. Forward Elimination of Unknowns: In this step, the unknown is eliminated in each equation starting with the first equation. This way, the equations are *reduced* to one equation and one unknown in each equation.
2. Back Substitution: In this step, starting from the last equation, each of the unknowns is found.

Forward Elimination of Unknowns:

In the first step of forward elimination, the first unknown, x_1 is eliminated from all rows below the first row. The first equation is selected as the pivot equation to eliminate x_1 . So, to eliminate x_1 in the second equation, one divides the first equation by a_{11} (hence called the pivot element) and then multiplies it by a_{21} . This is the same as multiplying the first equation by a_{21}/a_{11} to give

$$a_{21}x_1 + \frac{a_{21}}{a_{11}}a_{12}x_2 + \dots + \frac{a_{21}}{a_{11}}a_{1n}x_n = \frac{a_{21}}{a_{11}}b_1$$

Now, this equation can be subtracted from the second equation to give

$$\left(a_{22} - \frac{a_{21}}{a_{11}} a_{12} \right) x_2 + \dots + \left(a_{2n} - \frac{a_{21}}{a_{11}} a_{1n} \right) x_n = b_2 - \frac{a_{21}}{a_{11}} b_1$$

or

$$a'_{22} x_2 + \dots + a'_{2n} x_n = b'_2$$

where

$$\begin{aligned} a'_{22} &= a_{22} - \frac{a_{21}}{a_{11}} a_{12} \\ &\vdots \\ a'_{2n} &= a_{2n} - \frac{a_{21}}{a_{11}} a_{1n} \end{aligned}$$

This procedure of eliminating x_1 , is now repeated for the third equation to the n^{th} equation to reduce the set of equations as

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \dots + a'_{2n}x_n &= b'_2 \\ a'_{32}x_2 + a'_{33}x_3 + \dots + a'_{3n}x_n &= b'_3 \\ \cdot &\quad \cdot \quad \cdot \\ \cdot &\quad \cdot \quad \cdot \\ \cdot &\quad \cdot \quad \cdot \\ a'_{n2}x_2 + a'_{n3}x_3 + \dots + a'_{nn}x_n &= b'_n \end{aligned}$$

This is the end of the first step of forward elimination. Now for the second step of forward elimination, we start with the second equation as the pivot equation and a'_{22} as the pivot element. So, to eliminate x_2 in the third equation, one divides the second equation by a'_{22} (the pivot element) and then multiply it by a'_{32} . This is the same as multiplying the second equation by a'_{32} / a'_{22} and

subtracting it from the third equation. This makes the coefficient of x_2 zero in the third equation. The same procedure is now repeated for the fourth equation till the n^{th} equation to give

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \dots + a'_{2n}x_n &= b'_2 \\ a''_{33}x_3 + \dots + a''_{3n}x_n &= b''_3 \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ a''_{n3}x_3 + \dots + a''_{nn}x_n &= b''_n \end{aligned}$$

The next steps of forward elimination are conducted by using the third equation as a pivot equation and so on. That is, there will be a total of $n-1$ steps of forward elimination. At the end of $n-1$ steps of forward elimination, we get a set of equations that look like

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n &= b_1 \\ a'_{22}x_2 + a'_{23}x_3 + \dots + a'_{2n}x_n &= b'_2 \\ a''_{33}x_3 + \dots + a''_{3n}x_n &= b''_3 \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ a^{(n-1)}_{nn}x_n &= b^{(n-1)}_n \end{aligned}$$

Back Substitution:

Now the equations are solved starting from the last equation as it has only one unknown.

$$x_n = \frac{b_n^{(n-1)}}{a_{nn}^{(n-1)}}$$

Then the second last equation, that is the $(n-1)^{\text{th}}$ equation, has two unknowns: x_n and x_{n-1} , but is already known. This reduces the $(n-1)^{\text{th}}$ equation also to one unknown. Back substitution hence can be represented for all equations by the formula

$$x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{(i-1)}} \quad \text{for } i = n-1, n-2, \dots, 1$$

and

$$x_n = \frac{b_n^{(n-1)}}{a_{nn}^{(n-1)}}$$

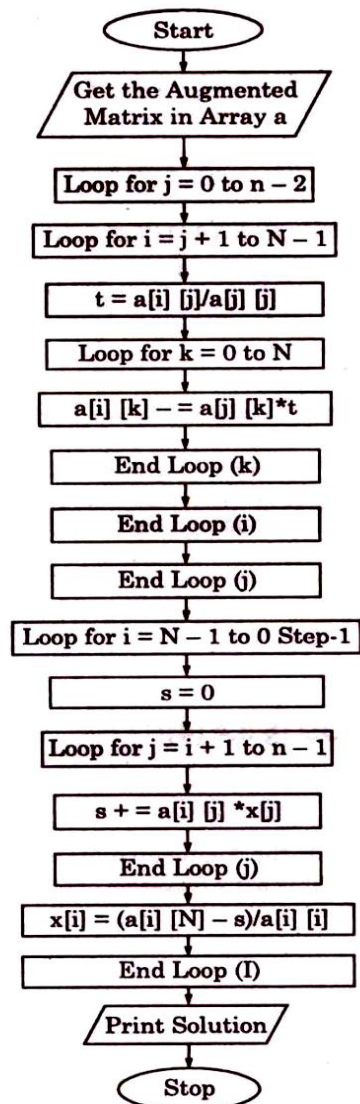
Pivoting

The primary reason that the foregoing technique is called “naïve” is that during both the elimination and the back-substitution phases, it is possible that a division by zero can occur.

Problems may also arise when the pivot element is close, rather than exactly equal, to zero because if the magnitude of the pivot element is small compared to the other elements, then round-off errors can be introduced. Therefore, before each row is normalized, it is advantageous to determine the coefficient with the largest absolute value in the column below the pivot element. The rows can then be switched so that the largest element is the pivot element. This is called *partial pivoting*.

If columns as well as rows are searched for the largest element and then switched, the procedure is called *complete pivoting*. Complete pivoting is rarely used because most of the improvement comes from partial pivoting. In addition, switching columns changes the order of the x 's and, consequently, adds significant and usually unjustified complexity to the computer program.

ALGORITHM



SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<math.h>

#include<stdlib.h>

main()

{

clrscr();

void gauss1 (int n, float a[10][10],float b[10], float x[10], int *status);

int status,n,i,j;

float a[10][10],b[10],x[10];

FILE *f;

FILE *ff;

printf("\n SOLUTION BY GAUSS ELEMINATION METHOD \n");

printf("\nWhat is the size of of the system (n)?\n");

scanf("%d",&n);

printf("\nInput Matrix from the file\n");

f=fopen("f1.dat","r");

ff=fopen("f2.dat","r");

for(i=1;i<=n;i++)

    for(j=1;j<=n;j++)

        fscanf(f,"%f",&a[i][j]);

for(i=1;i<=n;i++)

    fscanf(ff,"%f",&b[i]);

```

```

for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
            {
                printf("%f ",a[i][j]);
            }
        printf("\n");
    }

for(i=1;i<=n;i++)
    printf("\n%f",b[i]);

gauss1(n,a,b,x,&status);

if(status!=0)
{
    printf("\nSolution vector X\n");
    for(i=1;i<=n;i++)
        printf("%f ",x[i]);
    printf("\n");
}

else
{
    printf("\nSingular Matrix, reorder equations. \n");
}

}

void gauss1 (int n, float a[10][10],float b[10], float x[10], int *status)

```

```

{
int i,j,k;

float pivot, factor,sum;

for(k=1;k<=n;k++)
{
    //printf("\n%f",a[k][k]);

    pivot = a[k][k];

    if(pivot==0.000)
    {
        *status=0;

        return;
    }

    *status=1;

    for(i=k+1;i<=n;i++)
    {
        factor=a[i][k]/pivot;

        for(j=k;j<=n;j++)
        {
            a[i][j]=a[i][j]-factor*a[k][j];
        }

        b[i]=b[i]-factor*b[k];
    }
}

printf("\n");

```

```

for(i=1;i<=n;i++)
{
    for(j=1;j<=n;j++)
    {
        printf("%f ",a[i][j]);
    }
    printf("\n");
}

for(i=1;i<=n;i++)
{
    printf("\n%f",b[i]);
    x[n]=b[n]/a[n][n];
    for(k=n-1;k>=1;k--)
    {
        sum=0.0;
        for(j=k+1;j<=n;j++)
        {
            sum=sum+a[k][j]*x[j];
            x[k]=(b[k]-sum)/a[k][k];
        }
    }
    return;
}

```

GAUSS-JORDAN METHOD

The following row operations on the augmented matrix of a system produce the augmented matrix of an equivalent system, i.e., a system with the same solution as the original one.

- Interchange any two rows.
- Multiply each element of a row by a nonzero constant.
- Replace a row by the sum of itself and a constant multiple of another row of the matrix.

For these row operations, we will use the following notations.

- $R_i \leftrightarrow R_j$ means: Interchange row i and row j .
- αR_i means: Replace row i with α times row i .
- $R_i + \alpha R_j$ means: Replace row i with the sum of row i and α times row j .

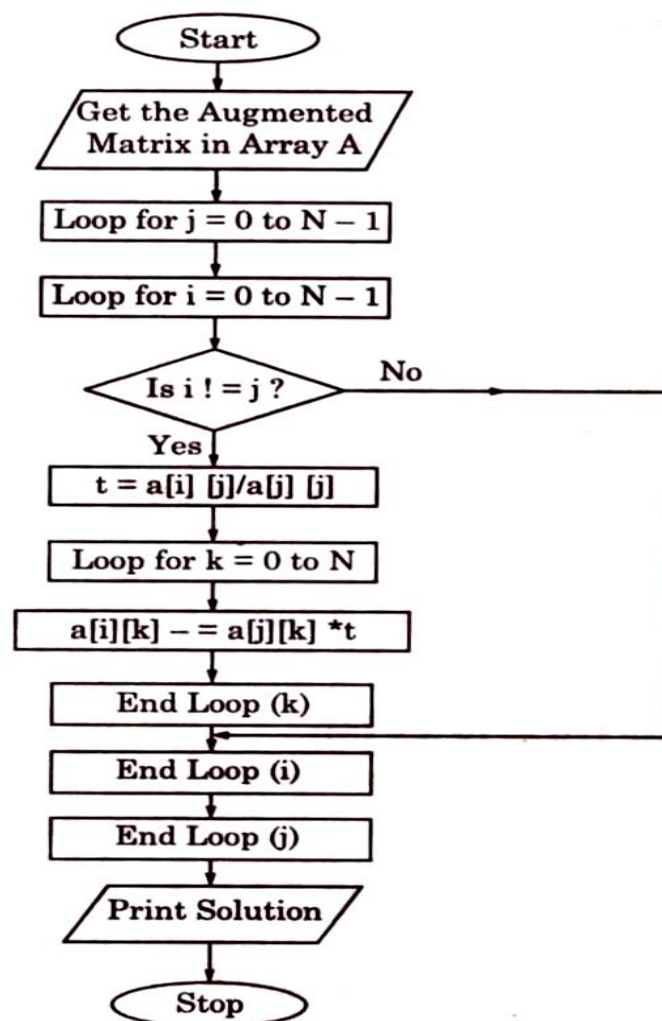
The Gauss-Jordan elimination method to solve a system of linear equations is described in the following steps.

- 1 Write the augmented matrix of the system.
- 2 Use row operations to transform the augmented matrix in the form described below, which is called the **reduced row echelon form** (RREF).
 - a) The rows (if any) consisting entirely of zeros are grouped together at the bottom of the matrix.
 - b) In each row that does not consist entirely of zeros, the leftmost nonzero element is a 1 (called a leading 1 or a pivot).
 - c) Each column that contains a leading 1 has zeros in all other entries.
 - d) The leading 1 in any row is to the left of any leading 1's in the rows below it.
- 3 Stop process in step 2 if you obtain a row whose elements are all zeros except the last one on the right. In that case, the system is inconsistent and

has no solutions. Otherwise, finish step 2 and read the solutions of the system from the final matrix.

Note: When doing step 2, row operations can be performed in any order. Try to choose row operations so that as few fractions as possible are carried through the computation. This makes calculation easier when working by hand.

ALGORITHM



SAMPLE PROGRAM

```

#include<stdio.h>

#define MX 20

#include<conio.h>

main()
{
float a[MX] [MX+1],m,p;
int i,j,k,n;
puts("\n how many equations?:");
scanf("%d",&n);
for(i=0;i<=n-1;i++)
{
printf("Give the coefficients of the equation no: %d\n",i+1);
for(j=0;j<=n;j++)
scanf("%f",&a[i][j]);
}
for(k=0;k<=n-1;k++)
{
for(i=0;i<=n-1;i++)
{
m=a[i][k]/a[k][k];
p=a[k][k];
for(j=k;j<=n;j++)
{

```



```
if(i==k)
a[i][j]=a[i][j]/p;
else
a[i][j]=a[i][j]-m*a[k][j];
}
}
}
for(i=0;i<=n-1;i++)
printf("\n X[%2d]=%5.2f",i,a[i][n]);
getch();
}
```

GAUSS-SEIDEL METHOD

In certain cases, such as when a system of equations is large, iterative methods of solving equations are more advantageous. Elimination methods, such as Gaussian elimination, are prone to large round-off errors for a large set of equations. Iterative methods, such as the Gauss-Seidel method, give the user control of the round-off error. Also, if the physics of the problem are well known, initial guesses needed in iterative methods can be made more judiciously leading to faster convergence. Given a general n set of equations and n unknowns, we have

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n &= c_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n &= c_2 \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ \cdot & \quad \cdot \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \dots + a_{nn}x_n &= c_n \end{aligned}$$

If the diagonal elements are non-zero, each equation is rewritten for the corresponding unknown, that is, the first equation is rewritten with x_1 on the left hand side, the second equation is rewritten with x_2 on the left hand side and so on as follows

$$\begin{aligned} x_1 &= \frac{c_1 - a_{12}x_2 - a_{13}x_3 - \dots - a_{1n}x_n}{a_{11}} \\ x_2 &= \frac{c_2 - a_{21}x_1 - a_{23}x_3 - \dots - a_{2n}x_n}{a_{22}} \\ &\vdots \\ &\vdots \\ x_{n-1} &= \frac{c_{n-1} - a_{n-1,1}x_1 - a_{n-1,2}x_2 - \dots - a_{n-1,n-2}x_{n-2} - a_{n-1,n}x_n}{a_{n-1,n-1}} \\ x_n &= \frac{c_n - a_{n1}x_1 - a_{n2}x_2 - \dots - a_{n,n-1}x_{n-1}}{a_{nn}} \end{aligned}$$

These equations can be rewritten in a summation form as

$$\begin{aligned}
 x_1 &= \frac{c_1 - \sum_{\substack{j=1 \\ j \neq 1}}^n a_{1j} x_j}{a_{11}} \\
 x_2 &= \frac{c_2 - \sum_{\substack{j=1 \\ j \neq 2}}^n a_{2j} x_j}{a_{22}} \\
 &\cdot \\
 &\cdot \\
 &\cdot \\
 x_{n-1} &= \frac{c_{n-1} - \sum_{\substack{j=1 \\ j \neq n-1}}^n a_{n-1,j} x_j}{a_{n-1,n-1}} \\
 x_n &= \frac{c_n - \sum_{\substack{j=1 \\ j \neq n}}^n a_{nj} x_j}{a_{nn}}
 \end{aligned}$$

Hence for any row i ,

$$x_i = \frac{c_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j}{a_{ii}}, i = 1, 2, \dots, n.$$

Now to find x_i 's, one assumes an initial guess for the x_i 's and then uses the rewritten equations to calculate the new estimates. Remember, one always uses the most recent estimates to calculate the next estimates, x_i . At the end of each iteration, one calculates the absolute relative approximate error for each x_i as

$$|\epsilon_a|_i = \left| \frac{x_i^{\text{new}} - x_i^{\text{old}}}{x_i^{\text{new}}} \right| \times 100$$

where x_i^{new} the recently obtained value of x_i , and x_i^{old} is the previous value of x_i . When the absolute relative approximate error for each x_i is less than the pre-specified tolerance, the iterations are stopped.

Convergence and Diagonal Dominance

Although the Gauss-Seidel method can also diverge, because it is designed for linear systems, its ability to converge is much more predictable than for fixed-point iteration of nonlinear equations. It can be shown that if the following condition holds, Gauss-Seidel will converge:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|$$

That is, the absolute value of the diagonal coefficient in each of the equations must be larger than the sum of the absolute values of the other coefficients in the equation. Such systems are said to be *diagonally dominant*. This criterion is sufficient but not necessary for convergence. That is, although the method may sometimes work if the above condition is not met, convergence is guaranteed if the condition is satisfied. Fortunately, many engineering and scientific problems of practical importance fulfill this requirement. Therefore, Gauss-Seidel represents a feasible approach to solve many problems in engineering and science.

SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<math.h>

void main()

{

    int n,i,j,k,l,nit;

    float a[10][10],b[10],x[10],x1[10],t;

    clrscr();

    printf("Give no. of variables ");

    scanf("%d", &n);

    for(i=0;i<=n;i++)

    {

        for(j=0;j<n;j++) {

            printf("%d ,%d  th coefficient wanted ", i+1 ,j+1);

            scanf("%f", &a[i][j]);

        }

        printf("%d th constant wanted ", i+1);

        scanf("%f", &b[i]);

    }

    /* Solution started*/

    for(i=0;i<=n;i++)

    {

        t = fabs(a[i][j]);

```

```

k = i ;
for(j=i+1;j<n; j++)
    if (fabs ( a[i][j] >t))
    {
        k = j;
        t = fabs ( a[j][i] );
    }
if( t< 1e - 5){
    printf(“Almost singular matrix, solution not possible\n”);
    exit(0);
}
if (i!= k)
    for(l=0;l<n;l++){
        t = a[i][l];
        a[i][l] = a[k][l];
        a[k][l]= t;
        t = b[i] ;
        b[i]= b[l];
        b[l] = t;
    }
}
for(i=0;i<=n;i++)
    x[i] = 0;
nit = 0;

```

```

while(i)
{
    for(i=0;i<=n;i++)
    {
        t = b[i] ;
        for(l=0;l<n;l++)
            if(l<i) t-=a[i][l] *x1[l];
        else
            if( l>i)t-= a[i][l] *x1[l];

        x1[i]= t/ a[i][i];
    }
    t = 0;
    for(l=0;l<n; l++)
        t += (x i [l] – x[l] ) * ( x1 [l] – x[l]) ;
    if (t< le -5) break;
    nit ++;
    if( nit >30)
    {
        printf(“Divergent\n”);
        exit(1);
    }
    for(l=0;l<n; l++)
        x [l] = x1 [l];

```

```
}  
printf(" Solution are \n") ;  
for( i=0 ; i< n ; i++)  
printf(" x[%d]= %f\n",i, x1[i]);  
getch();  
}
```


TRAPEZOIDAL RULE

The trapezoidal rule is based on the Newton-Cotes formula that if one approximates the integrand by an n^{th} order polynomial, then the integral of the function is approximated by the integral of that n^{th} order polynomial. Integrating polynomials is simple and is based on the calculus formula.

$$I = \int_a^b f(x) dx \cong \int_a^b f_n(x) dx$$

(1)

where $f_n(x)$ = a polynomial of the form

$$f_n(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} + a_nx^n$$

where n is the order of the polynomial. For example, in Fig. 1(a), a first-order polynomial (a straight line) is used as an approximation. In Fig. 1(b), a parabola is employed for the same purpose. The integral can also be approximated using a series of polynomials applied piecewise to the function or data over segments of constant length. For example, in Fig. 2, three straight-line segments are used to approximate the integral. Higher-order polynomials can be utilized for the same purpose.

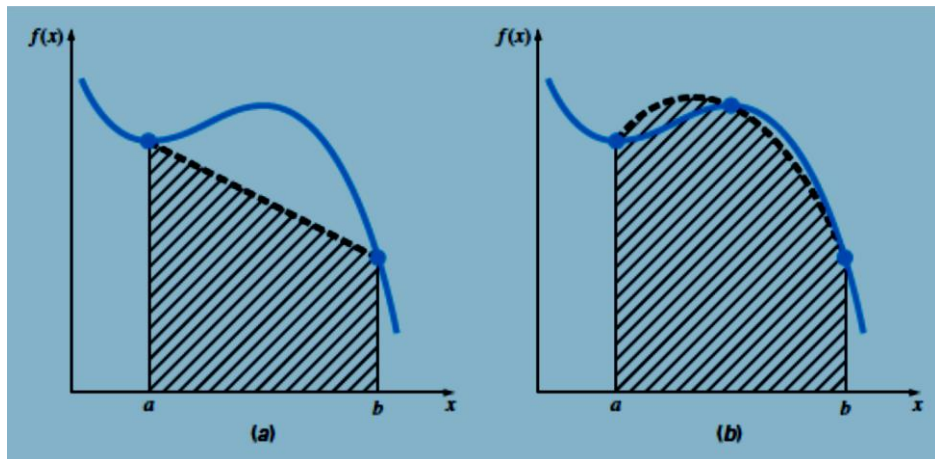


Figure 1. The approximation of an integral by the area under (a) a straight line and (b) a parabola.

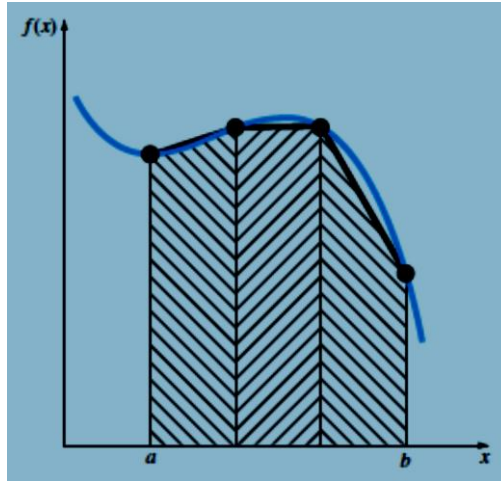


Figure 2. The approximation of an integral by the area under three straight-line segments.

The *trapezoidal rule* is the first of the Newton-Cotes closed integration formulas. It corresponds to the case where the polynomial in Eq. (1) is first-order:

$$I = \int_a^b \left[f(a) + \frac{f(b) - f(a)}{b - a}(x - a) \right] dx$$

The result of the integration is

$$I = (b - a) \frac{f(a) + f(b)}{2}$$

which is called the *trapezoidal rule*.

Geometrically, the trapezoidal rule is equivalent to approximating the area of the trapezoid under the straight line connecting $f(a)$ and $f(b)$ in Fig. 3. Recall from geometry that the formula for computing the area of a trapezoid is the height times the average of the bases. In our case, the concept is the same but the trapezoid is on its side. Therefore, the integral estimate can be represented as

$$I = \text{width} \times \text{average height}$$

or

$$I = (b - a) \times \text{average height} \quad (2)$$

where, for the trapezoidal rule, the average height is the average of the function values at the end points, or $[f(a) + f(b)]/2$.

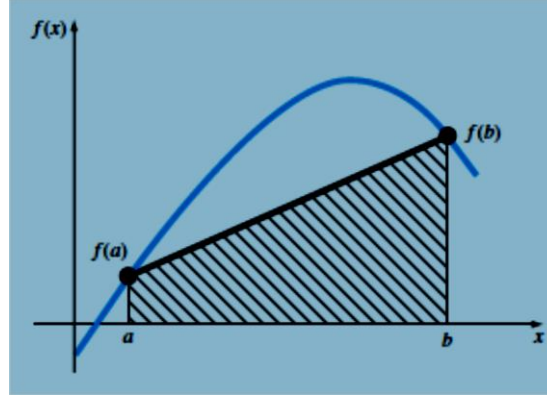


Figure 3. Graphical depiction of the trapezoidal rule.

An estimate for the local truncation error of a single application of the trapezoidal rule is

$$E_t = -\frac{1}{12} f''(\xi)(b-a)^3 \quad (3)$$

where ξ lies somewhere in the interval from a to b . Equation (3) indicates that if the function being integrated is linear, the trapezoidal rule will be exact because the second derivative of a straight line is zero. Otherwise, for functions with second- and higher-order derivatives (i.e., with curvature), some error can occur.

The Composite Trapezoidal Rule

One way to improve the accuracy of the trapezoidal rule is to divide the integration interval from a to b into a number of segments and apply the method to each segment (Fig. 4). The areas of individual segments can then be added to yield the integral for the entire interval. The resulting equations are called *composite*, or *multiple-segment, integration formulas*.

Figure 4 shows the general format and nomenclature we will use to characterize composite integrals. There are $n + 1$ equally spaced base points $(x_0, x_1, x_2, \dots, x_n)$. Consequently, there are n segments of equal width:

$$h = \frac{b-a}{n} \quad (4)$$

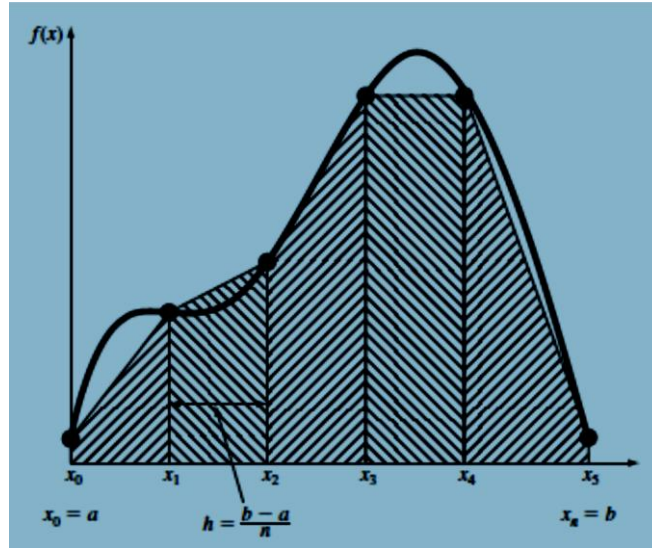


Figure 4. Composite trapezoidal rule.

If a and b are designated as x_0 and x_n , respectively, the total integral can be represented as

$$I = \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \cdots + \int_{x_{n-1}}^{x_n} f(x) dx$$

Substituting the trapezoidal rule for each integral yields

$$I = h \frac{f(x_0) + f(x_1)}{2} + h \frac{f(x_1) + f(x_2)}{2} + \cdots + h \frac{f(x_{n-1}) + f(x_n)}{2}$$

or, grouping terms:

$$I = \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right] \quad (5)$$

or, using Eq. (4) to express Eq. (5) in the general form of Eq. (2):

$$I = \underbrace{(b-a)}_{\text{Width}} \underbrace{\frac{f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n)}{2n}}_{\text{Average height}} \quad (6)$$

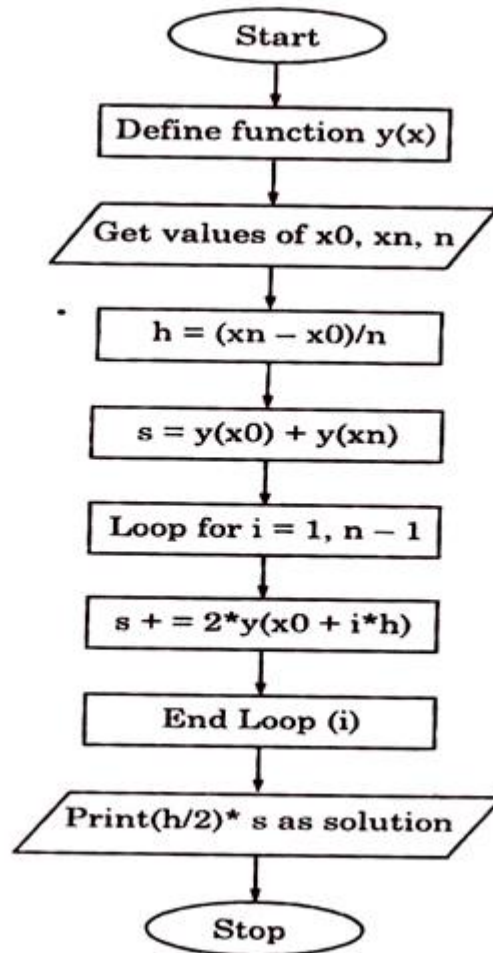
Because the summation of the coefficients of $f(x)$ in the numerator divided by $2n$ is equal to 1, the average height represents a weighted average of the

function values. According to Eq. (6), the interior points are given twice the weight of the two end points $f(x_0)$ and $f(x_n)$.

An error for the composite trapezoidal rule can be obtained by summing the individual errors for each segment to give

$$E_t = -\frac{(b-a)^3}{12n^3} \sum_{i=1}^n f''(\xi_i)$$

where $f''(\xi_i)$ is the second derivative at a point ξ_i located in segment i .

ALGORITHM

SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

float F(float x);

float trapzd(float a, float b,int n);

main()
{
clrscr();

int n;

float a,b,s;

printf("\na,b&n:");

scanf("%f%f%d",&a,&b,&n);

s=trapzd(a,b,n);

printf("\nIntegral=%f",s);

getche();
}

float trapzd(float a, float b,int n)
{
float x,tnm,sum,del;

static float s;

int it,j;

if(n==1){

return(s=0.5*(b-a)*(F(a)+F(b)));

}else{

```

```

    for(it=1,j=1;j<n-1;j++)it<=1;

    tnm=it;

    printf("\nit=%d",it);

    del=(b-a)/tnm;

    x=a+0.5*del;

    for(sum=0.0,j=1;j<=it;j++,x+=del) sum+=F(x);

    s=0.5*(s+(b-a)*sum/tnm) ;

    return s;

}

}

float F(float x)

{

float y;

```


SIMPSON'S RULE

The Simpson's rule is based on the Newton-Cotes formula that if one approximates the integrand by an n^{th} order polynomial, then the integral of the function is approximated by the integral of that n^{th} order polynomial. Integrating polynomials is simple and is based on the calculus formula.

$$I = \int_a^b f(x) dx \cong \int_a^b f_n(x) dx$$

(1)

where $f_n(x)$ = a polynomial of the form

$$f_n(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} + a_nx^n$$

where n is the order of the polynomial. For example, in Fig. 1(a), a first-order polynomial (a straight line) is used as an approximation. In Fig. 1(b), a parabola is employed for the same purpose. The integral can also be approximated using a series of polynomials applied piecewise to the function or data over segments of constant length. For example, in Fig. 2, three straight-line segments are used to approximate the integral. Higher-order polynomials can be utilized for the same purpose.

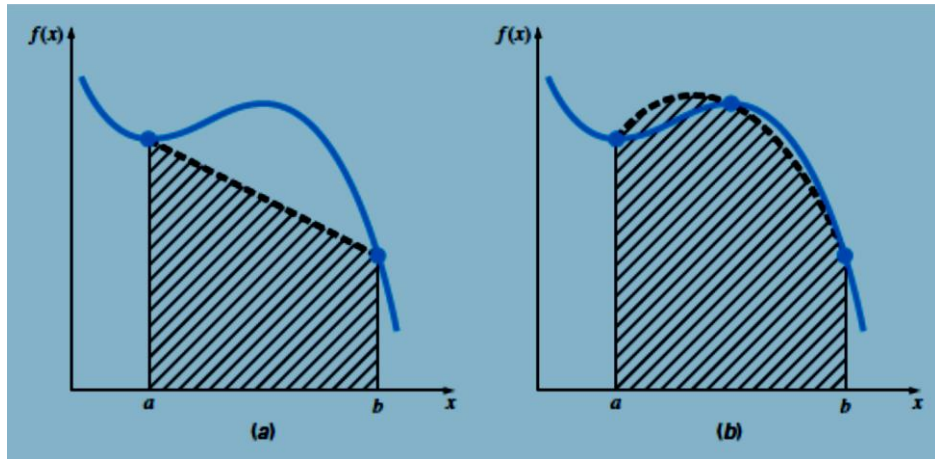


Figure 1. The approximation of an integral by the area under (a) a straight line and (b) a parabola.

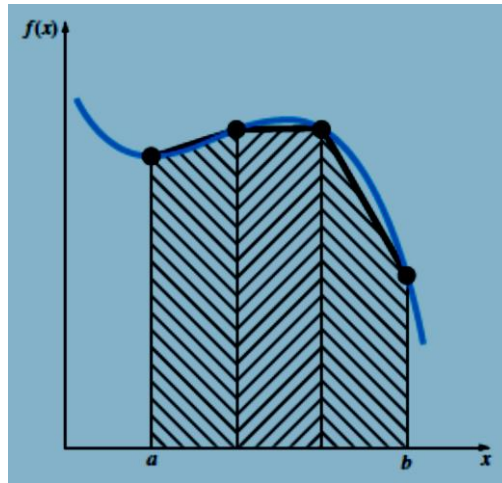


Figure 2. The approximation of an integral by the area under three straight-line segments.

Simpson's rule

Aside from applying the trapezoidal rule with finer segmentation, another way to obtain a more accurate estimate of an integral is to use higher-order polynomials to connect the points. For example, if there is an extra point midway between $f(a)$ and $f(b)$, the three points can be connected with a parabola (Fig. 3a). If there are two points equally spaced between $f(a)$ and $f(b)$, the four points can be connected with a third-order polynomial (Fig. 3b). The formulas that result from taking the integrals under these polynomials are called *Simpson's rules*.

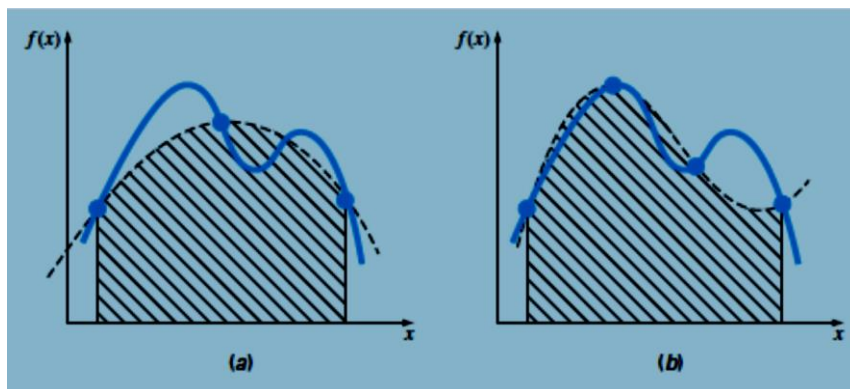


Figure 3. (a) Graphical depiction of Simpson's 1/3 rule: It consists of taking the area under a parabola connecting three points. (b) Graphical depiction of Simpson's 3/8 rule: It consists of taking the area under a cubic equation connecting four points.

Simpson's 1/3 Rule

Simpson's 1/3 rule corresponds to the case where the polynomial in Eq. (1) is secondorder:

$$I = \int_{x_0}^{x_2} \left[\frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2) \right] dx$$

where a and b are designated as x_0 and x_2 , respectively. The result of the integration is

$$I = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)] \quad (2)$$

where, for this case, $h = (b - a)/2$. This equation is known as *Simpson's 1/3 rule*. The label .1/3. stems from the fact that h is divided by 3 in Eq. (2).

$$I = (b - a) \frac{f(x_0) + 4f(x_1) + f(x_2)}{6} \quad (3)$$

where $a = x_0$, $b = x_2$, and x_1 = the point midway between a and b , which is given by $(a + b)/2$. Notice that, according to Eq. (3), the middle point is weighted by twothirds and the two end points by one-sixth.

It can be shown that a single-segment application of Simpson's 1/3 rule has a truncation error of

$$E_t = -\frac{1}{90} h^5 f^{(4)}(\xi)$$

or, because $h = (b - a)/2$:

$$E_t = -\frac{(b - a)^5}{2880} f^{(4)}(\xi) \quad (4)$$

where ξ lies somewhere in the interval from a to b . Thus, Simpson's 1/3 rule is more accurate than the trapezoidal rule. Rather than being proportional to the third derivative, the error is proportional to the fourth derivative. Consequently, Simpson's 1/3 rule is thirdorder accurate even though it is based on only three points. In other words, it yields exact results for cubic polynomials even though it is derived from a parabola!

The Composite Simpson's 1/3 Rule

Just as with the trapezoidal rule, Simpson's rule can be improved by dividing the integration interval into a number of segments of equal width (Fig. 4).

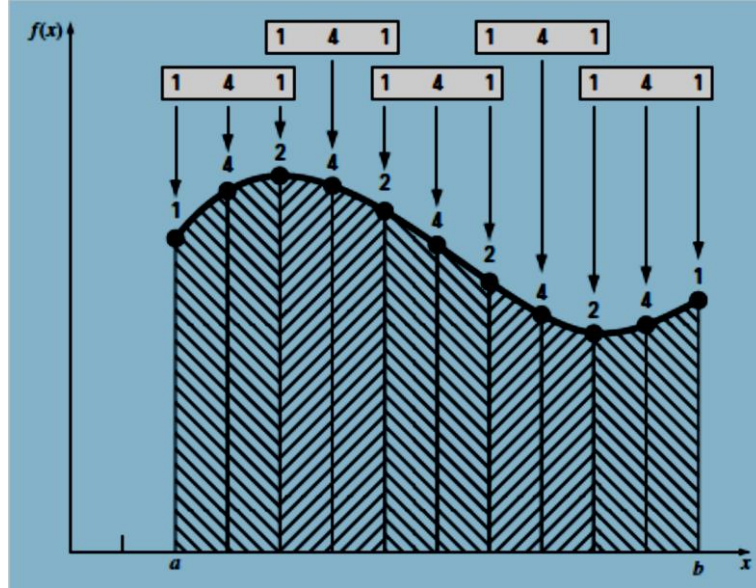


Figure 4. Composite Simpson's 1/3 rule. The relative weights are depicted above the function values. Note that the method can be employed only if the number of segments is even.

The total integral can be represented as

$$I = \int_{x_0}^{x_2} f(x) dx + \int_{x_2}^{x_4} f(x) dx + \cdots + \int_{x_{n-2}}^{x_n} f(x) dx$$

Substituting Simpson's 1/3 rule for each integral yields

$$I = 2h \frac{f(x_0) + 4f(x_1) + f(x_2)}{6} + 2h \frac{f(x_2) + 4f(x_3) + f(x_4)}{6} \\ + \cdots + 2h \frac{f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)}{6}$$

$$I = (b - a) \frac{f(x_0) + 4 \sum_{i=1,3,5}^{n-1} f(x_i) + 2 \sum_{j=2,4,6}^{n-2} f(x_j) + f(x_n)}{3n}$$

(5)

Notice that, as illustrated in Fig. 4, an even number of segments must be utilized to implement the method. In addition, the coefficients "4" and "2" in Eq. (5)

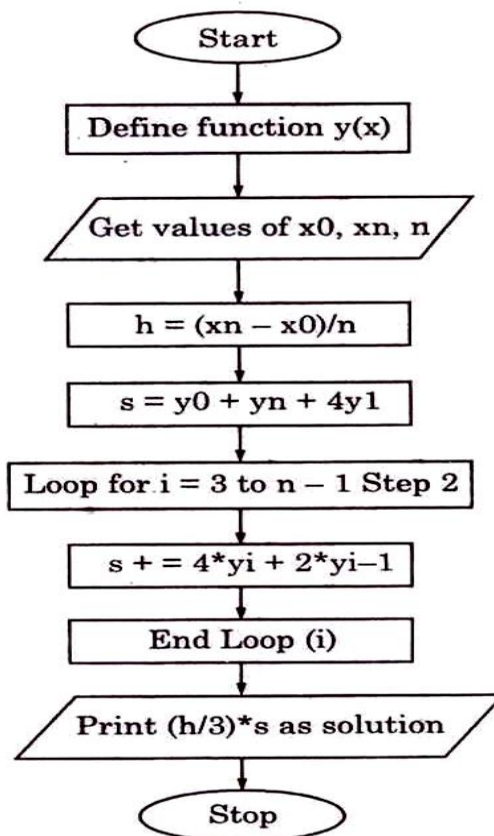
might seem peculiar at first glance. However, they follow naturally from Simpson's 1/3 rule. As illustrated in Fig. 4, the odd points represent the middle term for each application and hence carry the weight of four from Eq. (3). The even points are common to adjacent applications and hence are counted twice.

An error estimate for the composite Simpson's rule is obtained in the same fashion as for the trapezoidal rule by summing the individual errors for the segments and averaging the derivative to yield

$$E_a = -\frac{(b-a)^5}{180n^4} \bar{f}^{(4)}$$

where $f^{(4)}$ is the average fourth derivative for the interval.

ALGORITHM



SAMPLE PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

float simp(int n,float y[10],float sum)

{

int i; float s;

for(i=0;i<n-1;i++)

{

if(i%2==0)

sum+=2*y[i];

else

sum+=4*y[i];

}

s=sum;

return(s);

}

void main()

{

int n,i,j;

float s,sum,x[10],y[10],h;

clrscr();

sum=0;

printf("Enter the value for n");
```

```
scanf("%d",&n);  
printf("Enter the value for x");  
for (i=0;i<n;i++)  
scanf("%f",x[i]);  
printf("Enter the value of f(x)");  
for(i=0;i<n;i++)  
scanf("%f",y[i]);  
h=x[1]-x[0];  
sum+=y[0]+y[n-1];  
s=simp(n,y,sum);  
sum = s*h/3;  
printf("The integral is %f",sum);  
getch();  
return;  
}
```

GAUSS QUADRATURE FORMULAE

The trapezoidal rule is based on taking the area under the straight line connecting the function values at the ends of the integration interval. Because the trapezoidal rule must pass through the end points, there are cases such as Fig. 1a where the formula results in a large error.

Now, suppose that the constraint of fixed base points was removed and we were free to evaluate the area under a straight line joining any two points on the curve. By positioning these points wisely, we could define a straight line that would balance the positive and negative errors. Hence, as in Fig. 1b, we would arrive at an improved estimate of the integral. *Gauss quadrature* is the name for a class of techniques to implement such a strategy.

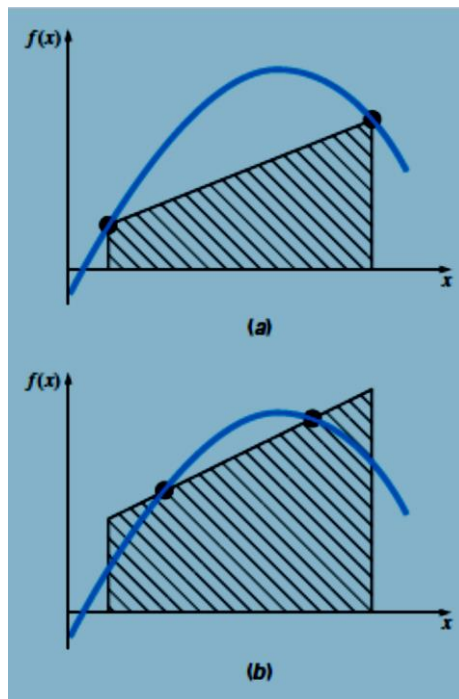


Figure 1.(a) Graphical depiction of the trapezoidal rule as the area under the straight line joining fixed end points. (b) An improved integral estimate obtained by taking the area under the straight line passing through two intermediate points. By positioning these points wisely, the positive and negative errors are better balanced, and an improved integral estimate results.

Derivation of the Two-Point Gauss-Legendre Formula

The object of Gauss quadrature is to determine the coefficients of an equation of the form

$$I \cong c_0 f(x_0) + c_1 f(x_1) \quad (1)$$

where the c 's = the unknown coefficients.

The function arguments x_0 and x_1 are not fixed at the end points, but are unknowns (Fig. 2). Thus, we now have a total of four unknowns that must be evaluated, and consequently, we require four conditions to determine them exactly.

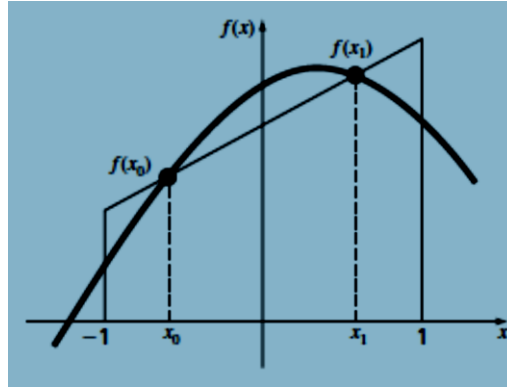


Figure 2. Graphical depiction of the unknown variables x_0 and x_1 for integration by Gauss quadrature.

We can obtain two of these conditions by assuming that Eq. (1) fits the integral of a constant and a linear function exactly. Then, to arrive at the other two conditions, we merely extend this reasoning by assuming that it also fits the integral of a parabolic ($y = x^2$) and a cubic ($y = x^3$) function. By doing this, we determine all four unknowns and in the bargain derive a linear two-point integration formula that is exact for cubics. The four equations to be solved are

$$c_0 + c_1 = \int_{-1}^1 1 dx = 2 \quad (2)$$

$$c_0 x_0 + c_1 x_1 = \int_{-1}^1 x dx = 0 \quad (3)$$

$$c_0 x_0^2 + c_1 x_1^2 = \int_{-1}^1 x^2 dx = \frac{2}{3} \quad (4)$$

$$c_0 x_0^3 + c_1 x_1^3 = \int_{-1}^1 x^3 dx = 0 \quad (5)$$

Equations (2) through (5) can be solved simultaneously for the four unknowns. First, solve Eq. (3) for c_1 and substitute the result into Eq. (5), which can be solved for

$$x_0^2 = x_1^2$$

Since x_0 and x_1 cannot be equal, this means that $x_0 = -x_1$. Substituting this result into Eq. (3) yields $c_0 = c_1$. Consequently from Eq. (2) it follows that

$$c_0 = c_1 = 1$$

Substituting these results into Eq. (4) gives

$$\begin{aligned} x_0 &= -\frac{1}{\sqrt{3}} = -0.5773503\dots \\ x_1 &= \frac{1}{\sqrt{3}} = 0.5773503\dots \end{aligned}$$

Therefore, the two-point Gauss-Legendre formula is

$$I = f\left(\frac{-1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) \quad (6)$$

Thus, we arrive at the interesting result that the simple addition of the function values at $x = -1/\sqrt{3}$ and $1/\sqrt{3}$ yields an integral estimate that is third-order accurate.

Notice that the integration limits in Eqs. (2) through (5) are from -1 to 1 . This was done to simplify the mathematics and to make the formulation as general as possible. A simple change of variable can be used to translate other limits of integration into this form. This is accomplished by assuming that a new variable x_d is related to the original variable x in a linear fashion, as in

$$x = a_1 + a_2 x_d \quad (7)$$

If the lower limit, $x = a$, corresponds to $x_d = -1$, these values can be substituted into Eq. (7) to yield

$$a = a_1 + a_2(-1) \quad (8)$$

Similarly, the upper limit, $x = b$, corresponds to $x_d = 1$, to give

$$b = a_1 + a_2(1) \tag{9}$$

Equations (8) and (9) can be solved simultaneously for

$$a_1 = \frac{b+a}{2} \quad \text{and} \quad a_2 = \frac{b-a}{2} \tag{10}$$

which can be substituted into Eq. (7) to yield

$$x = \frac{(b+a) + (b-a)x_d}{2} \tag{11}$$

This equation can be differentiated to give

$$dx = \frac{b-a}{2} dx_d \tag{12}$$

Equations (11) and (12) can be substituted for x and dx , respectively, in the equation to be integrated. These substitutions effectively transform the integration interval without changing the value of the integral.

SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<math.h>

float F(float x)

{

float y;

y=exp(-x*x);

return y;

}

float gausq(float a, float b)

{

float F(float x);

int j;

float xm,xr,dx,s;

static float x[]={0.0,0.1488743389,0.4333953941,0.6794095682,0.8650633666,

                                0.9739065285};

static float w[]={0.0,.2955242247,0.2692667193,0.2190863625,0.1494513491,

                                0.0666713443};

xm=0.5*(b+a);

xr=0.5*(b-a);

s=0;

for (j=1;j<=5;j++){

```

```

    dx=xr*x[j];
    s+=w[j]*(F((xm+dx))+F((xm-dx)));
    printf("\nw[j]=%f/(xm+dx)=%f/(xm-dx)=%f",w[j],(xm+dx),(xm-dx));
}

return s*=xr;

}

main()
{
    clrscr();
    float a,b,intg;
    printf("\nEnter a&b:");
    scanf("%f%f",&a,&b);
    intg=gausq(a,b);
    printf("\nIntegral=%f",intg);
}

```

TAYLOR'S SERIES METHOD

Consider the one dimensional initial value problem

$$y' = f(x, y), \quad y(x_0) = y_0$$

where

f is a function of two variables x and y and (x_0, y_0) is a known point on the solution curve.

If the existence of all higher order partial derivatives is assumed for y at $x = x_0$, then by Taylor series the value of y at any neighbouring point $x+h$ can be written as

$$y(x_0+h) = y(x_0) + h y'(x_0) + \frac{h^2}{2!} y''(x_0) + \frac{h^3}{3!} y'''(x_0) + \dots$$

where $'$ represents the derivative with respect to x . Since at x_0, y_0 is known, y' at x_0 can be found by computing $f(x_0, y_0)$. Similarly higher derivatives of y at x_0 also can be computed by making use of the relation $y' = f(x, y)$

$$y'' = f_x + f_y y'$$

$$y''' = f_{xx} + 2f_{xy}y' + f_{yy} y'^2 + f_y y''$$

and so on. Then

$$y(x_0+h) = y(x_0) + hf + \frac{h^2}{2!}(f_x + f_y y') + \frac{h^3}{3!}(f_{xx} + 2f_{xy}y' + f_{yy} y'^2 + f_y y'') + o(h^4)$$

Hence the value of y at any neighbouring point $x_0 + h$ can be obtained by summing the above infinite series. However, in any practical computation, the summation has to be terminated after some finite number of terms. If the series has been terminated after the p^{th} derivative term then the approximated formula is called the Taylor series approximation to y of order p and the error is of order $p+1$. The same can be repeated to obtain y at other points of x in the interval $[x_0, x_n]$ in a marching process.

Error in the approximation : The Taylor series method of order p has the property that the final global error is of order $o(h^{p+1})$; hence p can be chosen as large as necessary to make the error as small as desired. If the order p is

fixed, it is theoretically possible to a priori determine the size of **h** so that the final global error will be as small as desired. Since

$$E_p = (1/(p+1)!) h^{p+1} y^{p+1}(x+\theta h) \quad 0 < \theta < 1$$

Making use of finite differences, the **p+1th** derivative of **y** at **x+θh** can be approximated as

$$E_p = \frac{h^p (y^p(x+\theta h) - y^p(x))}{(p+1)!}$$

However, in practice one usually computes two sets of approximations using step sizes **h** and **h/2** and compares the solutions. For **p = 4**, **E₄ = c * h⁴** and the same with step size **h/2**, **E₄ = c * (h/2)⁴**, that is if the step size is halved the error is reduced by an order of **1/16**.

Algorithm

Specify **x₀**, **x_n**, **y₀**, **h**

((**x₀**, **y₀**) Initial point,

x_n point where the solution is required

h the step length to be used in the marching process)

Repeat

compute **f(x_i, y_i)**, **f'(x_i, y_i)**, **f''(x_i, y_i)** . . .

compute **y(x_i+h) = y(x_i) + h f(x_i, y_i) + h²/2 f'(x_i, y_i) + h³/3! f''(x_i, y_i) + . . .**

x_i = x_i + h

until **x_i = x_n**

SAMPLE PROGRAM

//SOLUTION OF $Y'=X^2+Y^2$, $Y(X)=0$ WHEN $X=0$.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<math.h>
```

```
#include<stdlib.h>
```

//FUNCTION FOR RETURNING VALUES OF 1ST,2ND & 3RD DIFFERENTIALS AT $X=0$.

```
void derivs(float x0,float y0,float *dfx,float *d2fx,float *d3fx)
```

```
{
```

```
float x=x0,y=y0;
```

```
*dfx=pow(x,4)+1.0;
```

```
*d2fx= 4.0*pow(x,3);
```

```
*d3fx=4.0*3.0*pow(x,2);
```

```
}
```

//FUNCTION FOR RETURNING THE VALUE OF Y AT THE SPECIFIED VALUE OF X

```
void taylor(float x0,float y0,float xp,float incr)
```

```
{
```

```
FILE *f;
```

```
f=fopen("f1.dat","w+");
```

```
float h,x,y,dfx,d2fx,d3fx,y1;
```

```
int i,iter;
```

```
h=incr,x=x0,y=y0;
```

```
while(x<=xp)
```



```

{
    derivs(x,y,&dfx,&d2fx,&d3fx);
    y1=y0+h*dfx+h*h*d2fx/2.0+h*h*h*d3fx/6.0;
    printf("\nValue of yn at x:%f=%f",x,y);
    fprintf(f,"\n%f %f",x,y);
    x=x+h;

    if(x>xp){x=x-h;printf("\nValue of yn at x:%f=%f",x,y);y=y0=y1;return;}
    else
        y=y0=y1;
}

fclose(f);

printf("\nValue of yn at x:%f=%f",x,y);
}

main()
{
    clrscr();
    float x0=0,y0=0.0,incr=.01,xp=1.1;
    taylor(x0,y0,xp,incr);
    getch();
}

```

RUNGE-KUTTA METHOD

Runge-Kutta method here after called as RK method is the generalization of the concept used in Modified Euler's method.

In Modified Eulers method the slope of the solution curve has been approximated with the slopes of the curve at the end points of the each sub interval in computing the solution. The natural generalization of this concept is computing the slope by taking a weighted average of the slopes taken at more number of points in each sub interval. However, the implementation of the scheme differs from Modified Eulers method so that the developed algorithm is explicit in nature. The final form of the scheme is of the form

$$\mathbf{y}_{i+1} = \mathbf{y}_i + (\text{weighted average of the slopes}) \quad \text{for } i = 0, 1, 2 \dots$$

where $\mathbf{h}$ is the step length and $\mathbf{y}_i$ and $\mathbf{y}_{i+1}$ are the values of $\mathbf{y}$ at $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ respectively.

In genaral, the slope is computed at various points $\mathbf{x}_s$ in each sub interval $[\mathbf{x}_i, \mathbf{x}_{i+1}]$ and multiplied them with the step length $\mathbf{h}$ and then weighted average of it is then added to $\mathbf{y}_i$ to compute $\mathbf{y}_{i+1}$. Thus the RK method with $\mathbf{v}$ slopes called as $\mathbf{v}$ -stage RKmethod can be written as

$$\mathbf{K}_1 = \mathbf{h} f(\mathbf{x}_i, \mathbf{y}_i)$$

$$\mathbf{K}_2 = \mathbf{h} f(\mathbf{x}_i + \mathbf{c}_2 \mathbf{h}, \mathbf{y}_i + \mathbf{a}_{21} \mathbf{K}_1)$$

$$\mathbf{K}_3 = \mathbf{h} f(\mathbf{x}_i + \mathbf{c}_3 \mathbf{h}, \mathbf{y}_i + \mathbf{a}_{31} \mathbf{K}_1 + \mathbf{a}_{32} \mathbf{K}_2)$$

...

...

...

$$\mathbf{K}_v = \mathbf{h} f(\mathbf{x}_i + \mathbf{c}_v \mathbf{h}, \mathbf{y}_i + \mathbf{a}_{v1} \mathbf{K}_1 + \mathbf{a}_{v2} \mathbf{K}_2 + \dots + \mathbf{a}_{vv-1} \mathbf{K}_{v-1})$$

and

$$\mathbf{y}_{i+1} = \mathbf{y}_i + (\mathbf{W}_1 \mathbf{K}_1 + \mathbf{W}_2 \mathbf{K}_2 + \dots + \mathbf{W}_v \mathbf{K}_v) \quad \text{for } i = 0, 1, 2 \dots$$

To determine the parameters $\mathbf{c}$'s, $\mathbf{a}$'s and $\mathbf{W}$'s in the above equation, $\mathbf{y}_{i+1}$ defined in the scheme is expanded interms of steplength $\mathbf{h}$ and the resultant equation is then compared with Taylor series expansion of the solution of the differential equation upto a certain number of terms say $\mathbf{p}$. Then the $\mathbf{v}$ -stage RK method

will be of order p or is an p^{th} order RK method. Here for any $v > 4$ the maximum possible order p of the RK method is always less than v . However, for any v less than or equal to 4, it is possible to derive an RK method of order $p = v$. Now, consider the case $v = 2$ to derive the 2-stage RK method. For this

$$\begin{aligned} K_1 &= h f(x_i, y_i) \\ K_2 &= h f(x_i + c_2 h, y_i + a_{21} K_1) \\ y_{i+1} &= y_i + (W_1 K_1 + W_2 K_2) \end{aligned} \quad \text{for } i = 0, 1, 2, \dots$$

Now by Taylor series expansion

$$\begin{aligned} y(x_{i+1}) &= y(x_i) + h y'(x_i) + \frac{h^2}{2!} y''(x_i) + \frac{h^3}{3!} y'''(x_i) + o(h^4) \\ &= y(x_i) + h f + \frac{h^2}{2!} (f_x + f_y f) + \frac{h^3}{3!} (f_{xx} + 2f_{xy} f + f_{yy} f^2 + f_y (f_x + f_y f)) + o(h^4) \end{aligned}$$

Also

$$\begin{aligned} K_1 &= h f_i \\ K_2 &= h f(x_i + c_2 h, y_i + a_{21} K_1) \\ &= h (f_i + c_2 h f_x + a_{21} K_1 f_y + (c_2 h)^2 f_{xx} / 2! + (a_{21} K_1)^2 f_{yy} / 2! + c_2 h a_{21} K_1 f_{xy} + o(h^4)) \\ &= h (f_i + c_2 h f_x + a_{21} h f_i f_y + (c_2 h)^2 f_{xx} / 2! + (a_{21} h f_i)^2 f_{yy} / 2! + c_2 h a_{21} h f_i f_{xy} + o(h^4)) \\ y_{i+1} &= y_i + (W_1 + W_2) h f_i + h^2 (W_2 c_2 f_x + W_2 a_{21} f_y) + h^3 W_2 (c_2^2 a_{21} f_{xy} + a_{21}^2 f_y^2) / 2 + o(h^4) \end{aligned}$$

Now by comparing the equal powers of h in y_{i+1} and $y(x_{i+1})$ we get

$$W_1 + W_2 = 1 \quad c_2 W_2 = 1/2 \quad \text{and} \quad a_{21} W_2 = 1/2$$

The solution of this system is

$$a_{21} = c_2, \quad W_2 = 1/(2c_2) \quad \text{and} \quad W_1 = 1 - 1/(2c_2)$$

where c_2 is any arbitrary constant not equal to zero. For these values of a_{21} , W_2 , W_1 , since 2-stage RK method compares with Taylor series upto h^2 for any value of c_2 the 2-stage RK method is of order two and hence this scheme is denoted in many text books as a second order RK method. Now, to give some numerical values to a_{21} , W_2 , W_1 first the value c_2 has to be fixed. Generally the value of c_2 is fixed such that the values of a_{21} , W_2 , W_1 are integers or some real numbers which are easy to remember. Two of such cases are $c_2 = 1/2$ and $c_2 = 1$.

Case (i): $c_2 = 1/2$ $\Rightarrow$ $a_{21} = 1/2$, $W_2 = 1$, $W_1 = 0$. The corresponding 2-stage (second order) RK method is

$$\begin{aligned} K_1 &= h f(x_i, y_i) \\ K_2 &= h f(x_i + h/2, y_i + K_1/2) \\ y_{i+1} &= y_i + (K_2) \end{aligned} \quad \text{for } i = 0, 1, 2 \dots$$

or equivalently

$$y_{i+1} = y_i + h f(x_i + h/2, y_i + h f(x_i, y_i)/2) \quad \text{for } i = 0, 1, 2 \dots$$

Which is nothing but Eulers method with step length $h = 1/2$.

Case (ii): $c_2 = 1 \rightarrow a_{21} = 2$, $W_2 = W_1 = 1/2$. The corresponding 2-stage (second order) RK method is

$$\begin{aligned} K_1 &= h f(x_i, y_i) \\ K_2 &= h f(x_i + h, y_i + K_1) \\ y_{i+1} &= y_i + (K_1 + K_2)/2 \end{aligned} \quad \text{for } i = 0, 1, 2 \dots$$

or equivalently

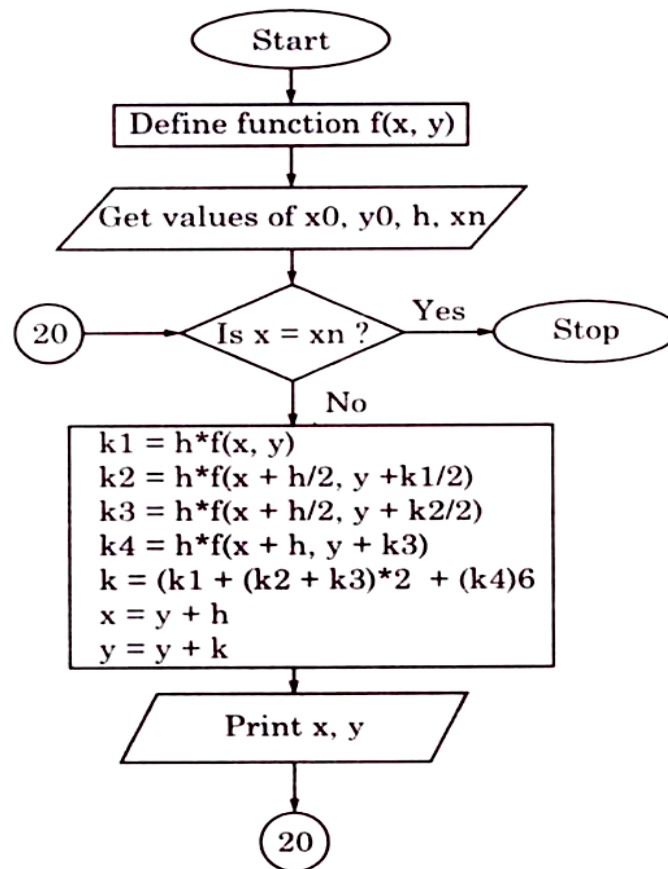
$$y_{i+1} = y_i + .5 h (f(x_i, y_i) + f(x_i + h, y_i + h f(x_i, y_i))) \quad \text{for } i = 0, 1, 2 \dots$$

Which is nothing but the Modified Eulers method. Following the same procedure one can develop the higher order RK methods by giving various values to v and comparing the obtained y_{i+1} with the same obtained by Taylor series method. Classical RK methods of order three and four are

RK method of order three ($v = 3$)	$K_1 = h f(x_i, y_i)$
	$K_2 = h f(x_i + h/2, y_i + K_1/2)$
	$K_3 = h f(x_i + h, y_i - K_1 + 2K_2)$
	$y_{i+1} = y_i + (K_1 + 4K_2 + K_3)/6$

RK method of order three ($v = 4$)	$K_1 = h f(x_i, y_i)$
	$K_2 = h f(x_i + h/2, y_i + K_1/2)$
	$K_3 = h f(x_i + h/2, y_i + K_2/2)$
	$K_4 = h f(x_i + h, y_i + K_3)$
	$y_{i+1} = y_i + (K_1 + 2K_2 + 2K_3 + K_4)/6$

ALGORITHM



SAMPLE PROGRAM

```

#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<math.h>

float F(float x,float y)

{

float f;

f=1.0+y*y;

return (f);

}

void rk4(float x0,float y0,float xn,float h)

```

```

{
int i,n;

float k1,k2,k3,k4;

n=(int)((xn-x0)/h+0.5);

for(i=1;i<=n;i++)
{
    k1=F(x0,y0);
    k2=F(x0+0.5*h,y0+0.5*k1*h);
    k3=F(x0+0.5*h,y0+0.5*k2*h);
    k4=F(x0+h,y0+k3*h);
    x0=x0+h;
    y0=y0+(k1+2.0*k2+2.0*k3+k4)*h/6.0;
    printf("\nValue of y at  xn=%f:  yn=%f",x0,y0);
}

printf("\nValue of y at x:xn=%f",y0);
}

main()
{
clrscr();

float x0,y0,h,xn;

x0=0.0,y0=0.0,xn=.4,h=0.01;

rk4(x0,y0,xn,h);

getch();
}

```

